

99. Qual è il tipo della seguente espressione:

```
fun f x y = x y;
```

- ☐ fun: 'a->'b
- ☐ fun: 'a->'b->string->string
- ☒ **fun: ('a->'b)->'a->'b**
- ☐ fun: int->int
- ☐ fun: string->string->string->string

100. Qual è il tipo della seguente espressione:

```
List.map print string;
```

- ☐ fun : string list -> string list
- ☒ **fun : string list -> unit list**
- ☐ fun : string list -> string
- ☐ fun : string -> string list
- ☐ fun: string -> unit
- ☐ fun: 'a list -> 'b list

101. Qual è il tipo della seguente espressione:

```
let f1 l = List.map (fun x y -> x,y) l;
```

- ☐ f1:'a list->('b -> 'a -> 'b) list
- ☐ f1:'a list-> 'b -> ('a * 'b)
- ☐ fun x y -> x,y : 'a -> 'b -> 'a * 'b
- ☐ fun x y -> x,y : 'a -> 'b -> 'a -> 'b
- ☒ **f1:'a list->('b -> 'a * 'b) list**

102. Qual è il tipo della seguente espressione:

```
(fn x => x) 12;
```

- ☒ **12: int**
- ☐ 'a: 12
- ☐ string: "12"
- ☐ fn: 'a -> 'a
- ☐ Nessuna delle altre risposte

103. Qual è il tipo della seguente espressione:

```
fun f1 x = [x,x];
```

- ☐ f1 = fn: 'a list -> 'a list
- ☐ f1 = fn: 'a list -> 'b list

- ☐ f1 = fn: 'a -> 'b
- ☒ **f1 = fn: 'a -> 'a list**
- ☐ f1 = fn: 'a -> 'a
- ☐ f1 = fn: string -> string

104. Qual è il tipo della seguente espressione:

```
fun f2 x y = (x @ (y x)) ;
```

- ☐ fun: 'a -> 'a -> 'a -> 'a
- ☒ **fun: 'a list -> ('a list -> 'a list) -> 'a list**
- ☐ fun: 'a list -> 'a list -> 'a list
- ☐ fun: 'a list -> 'a list -> 'b list
- ☐ fun: 'a -> ('a list -> 'a list) -> 'a list

105. Qual è il tipo della seguente espressione:

```
fun f3 x = List.map x;
```

- ☐ fun : ('a list -> 'b list) -> 'a list -> 'b list
- ☐ fun : ('a -> 'b) -> 'a -> 'b
- ☐ fun : ('a -> 'a) -> 'a list -> 'a list
- ☐ fun : 'a list -> 'b list
- ☒ **fun : ('a -> 'b) -> 'a list -> 'b list**

106. Qual è il tipo della seguente espressione:

```
fn x => fn y => x+y;
```

- ☐ fn: int -> int
- ☐ fn: 'a -> 'a -> 'a
- ☒ **fn: int -> int -> int**
- ☐ fn: 'a -> 'a
- ☐ fn: int -> int -> 'a
- ☐ Nessuna delle altre risposte

107. Qual è il tipo della seguente espressione:

```
fun funzione x y z = x ^ y ^ z
```

- ☐ fn: string -> string
- ☐ fn: 'a -> 'a
- ☐ fn: 'a -> 'a -> 'a -> 'a
- ☐ fn: string -> string -> string -> int
- ☐ Nessuna delle altre risposte
- ☒ **fn: string -> string -> string -> string**

108. Qual è il tipo della seguente espressione:

```
fun c a1 a2 = a1::a2;
```

- ☐ fn: 'a list -> 'a list -> 'a list
- ☒ **fn: 'a -> 'a list -> 'a list**
- ☐ fn: 'a -> 'a -> 'a
- ☐ fn: 'a -> 'a list
- ☐ fn: 'a -> 'a -> 'a list

109. Qual è il tipo della seguente espressione:

```
fn 11 => fn 12 => 11@12;
```

- ☒ **fn: 'a list -> 'a list -> 'a list**
- ☐ fn: 'a -> 'a list -> 'a list
- ☐ fn: 'a -> 'a -> 'a
- ☐ fn: 'a -> 'a list
- ☐ fn: 'a -> 'a -> 'a list

110. Qual è il tipo della seguente espressione:

```
fn a => fn b => fn c => (a * b) / c;
```

- ☐ fn: int -> int -> int -> int
- ☐ fn: 'a -> 'a -> 'a -> 'a
- ☐ fn: int -> int -> int -> real
- ☒ **fn: real -> real -> real -> real**
- ☐ fn: int -> int -> real -> real

111. Qual è il tipo della seguente espressione:

```
fun f [] = (7.0; print("ab")) | f (x::xs) = (2.3; print("cd"));
```

- ☒ **val f = fn: 'a list -> unit**
- ☐ 'a list -> int
- ☐ 'a tuple
- ☐ val f = fn: 'a list -> real
- ☐ real list -> string

112. Qual è l'output del seguente codice:

```
List.partition(fn x => x > 0) (List.take ([~1,1,~2,~2,~3,3],4));
```

- ☐ val it = ([1,3], [~1, ~2, ~2, ~3]): int list * int list
- ☐ val it = [1,3]: int list
- ☐ val it = [1]: int list

- ☐ **val it = ([1], [~1, ~2, ~2]): int list * int list**
- ☐ val it = ([], [~2]): int list * int list
- ☐ val it = []: int list

113. Qual è il tipo della seguente espressione:

fn a => fn b => a = b;

- ☐ **fn: 'a -> 'a -> bool**
- ☐ fn: int -> int -> bool
- ☐ fn: string -> string -> bool
- ☐ fn: bool -> bool -> bool
- ☐ fn: 'a -> 'a -> bool